كلية العلوم و التقنيات بطنجة
Faculté des Sciences et Techniques de Tanger

جامعة عبد المالك السعدي
Université Abdelmalek Essaâdi

Faculté des Sciences et Techniques de Tanger (F.S.T.T)

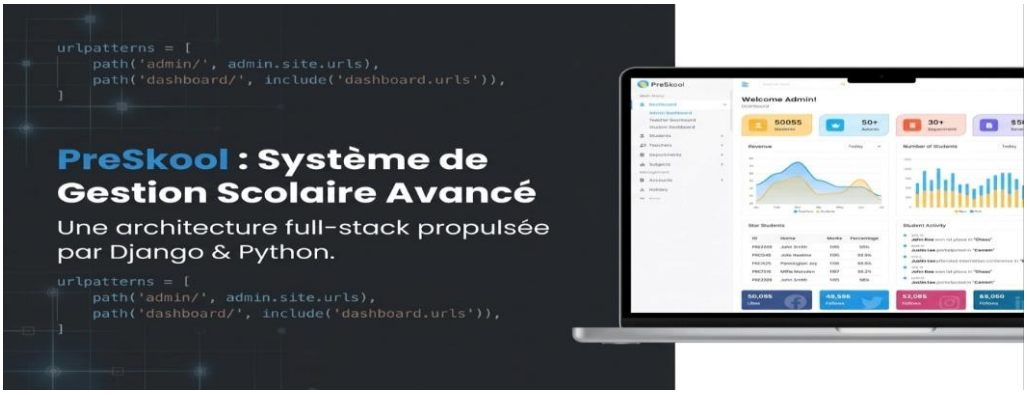
Département : Génie Informatique

LST – IDAI : Developpement Web avancé Back end (Python)

RAPPORT DE PROJET

Application Web : PreSkool Academy

“Système Intégré de Gestion Scolaire & Services Académiques Digitaux”



- **Encadré par :**

  Pr. Sara Ahsain

- **Réalisé par :**

  Dany Homam

  Elhajioui Houssam

Année Universitaire : 2025 / 2026

REMERCIEMENTS

En preambule a ce rapport technique, je souhaite exprimer mes plus vifs remerciements a toutes les personnes qui ont contribue, de pres ou de loin, a l'aboutissement de ce projet de fin de module.

Je tiens avant tout a temoigner ma profonde reconnaissance et mon immense respect a mon enseignante, Mme. Sara AHSAIN, pour l'honneur qu'elle m'a fait en encadrant ce travail. Ses conseils precieux, sa rigueur methodologique et sa disponibilite constante ont ete des piliers essentiels dans ma maitrise des technologies web avancees. Au-dela des aspects purement techniques, son exigence professionnelle m'a permis de developper une vision structuree de l'architecture logicielle.

Mes remerciements s'etendent a l'ensemble du departement Genie Informatique de la Faculte des Sciences et Techniques de Tanger (FSTT), pour la qualite de la formation dispensee et la richesse des ressources mises a notre disposition.

Enfin, j'adresse une pensee particuliere a mes proches pour leur soutien moral indefectible, qui m'a insuffle la perseverance necessaire pour mener a bien cette realisation complexe au sein de l'ecosysteme Django.

SOMMAIRE

1. Contexte du Projet 4
- 1.1 Presentation de l'Etablissement Fictif 4
- 1.2 Problematique Resolue 4

2. Objectifs du Projet 5
- 2.1 Objectifs Fonctionnels 5
- 2.2 Objectifs Techniques 5

3. Architecture Generale 6
- 3.1 Structure des Applications Django 6
- 3.2 Flux d'Authentification et de Navigation 6

4. Diagramme de Classes 7
- 4.1 Entites et Relations UML 7
- 4.2 Description des Entites Principales 7

5. Choix Techniques 9
5.1 Technologies et Justifications 9
5.2 Justification du Choix de MySQL 10

6. Description des Fonctionnalites Principales 10
6.1 Authentification et Gestion des Roles 10
6.2 Console d'Administration 10
6.3 Gestion des Etudiants et Parents 11
6.4 Gestion des Enseignants et Departements 12
6.5 Emplois du Temps 12
6.6 Gestion des Examens et des Notes 13
6.7 Gestion des Frais de Scolarite 13
6.8 Gestion des Jours Feries et Conges 14
6.9 Espace Etudiant et Tableau de Bord 14
6.10 Demande et Validation de Certificats 15

7. Diagrammes UML Complementaires 16
7.1 Diagramme de Sequence 16

8. Difficultes Rencontrees 17
8.1 Configuration de l'Environnement de Developpement 17
8.2 Gestion des Relations ORM Complexes 17
8.3 Generation de PDF 18
8.4 Securite et Controle d'Acces 18

9. Ameliorations Possibles 18
9.1 Fonctionnalites Non Implementees 18
9.2 Optimisations Techniques 19
9.3 Conclusion 19

10. Lien de GitHub 20

1. Contexte du Projet

1.1 Presentation de l'Etablissement Fictif

PreSkool Academy est un etablissement universitaire fictif a taille humaine, accueillant plusieurs centaines d'etudiants repartis en differentes filieres scientifiques et technologiques. L'etablissement dispose de plusieurs departements academiques (Mathematiques, Physique, Informatique, etc.), d'un corps enseignant diversifie et d'une administration centrale chargee de la gestion quotidienne.

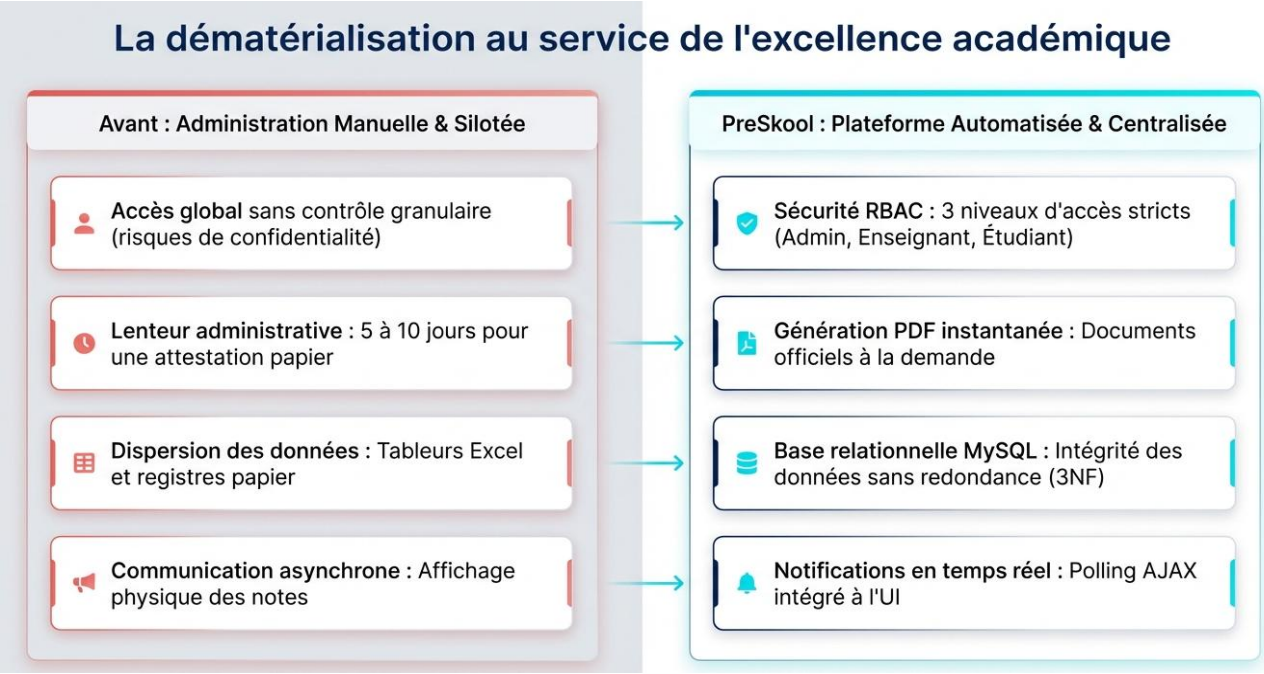
Dans le cadre de notre projet de fin de module, nous avons imagine cet etablissement confronte aux defis classiques de toute institution educative moderne : gestion manuelle chronophage, manque de visibilite en temps reel sur les performances academiques et difficulte a centraliser l'information.

1.2 Problematique Resolue

L'etablissement PreSkool Academy faisait face a plusieurs problemes operationnels majeurs :

- Dispersion des donnees : les informations sur les etudiants, les enseignants, les notes et les emplois du temps etaient gerees dans des fichiers Excel ou sur papier, rendant toute consultation rapide impossible.
- Absence de communication centralisee : les etudiants ne pouvaient pas consulter leurs resultats, leurs emplois du temps ou leurs examens sans passer par l'administration.
- Manque de traçabilite : les modifications de notes, les demandes de certificats ou les frais de scolarite n'etaient pas tracables.
- Gestion de documents fastidieuse : l'emission d'attestations de scolarite ou de releves de notes necessitait des demarches administratives longues et manuelles.

La solution proposee est une application web complete developpee avec le framework Django, permettant de centraliser l'ensemble des operations academiques sur une plateforme unique, securisee et accessible selon les roles (Administrateur, Enseignant, Etudiant).



2. Objectifs du Projet

L'application PreSkool Academy a ete concue pour repondre a un ensemble d'objectifs precis, a la fois techniques et fonctionnels.

2.1 Objectifs Fonctionnels

- Centraliser la gestion des etudiants, des enseignants et du personnel administratif.
- Permettre la consultation et la saisie des notes par les enseignants, avec un systeme de validation par l'administration.
- Offrir aux etudiants un espace personnel pour consulter leurs resultats, leur emploi du temps et leurs examens planifies.
- Automatiser la generation de documents officiels (attestations de scolarite, releves de notes, certificats de reussite).
- Gerer les frais de scolarite, les jours feries, les evenements et la bibliotheque.
- Proposer un tableau de bord analytique pour chaque type d'utilisateur.

2.2 Objectifs Techniques

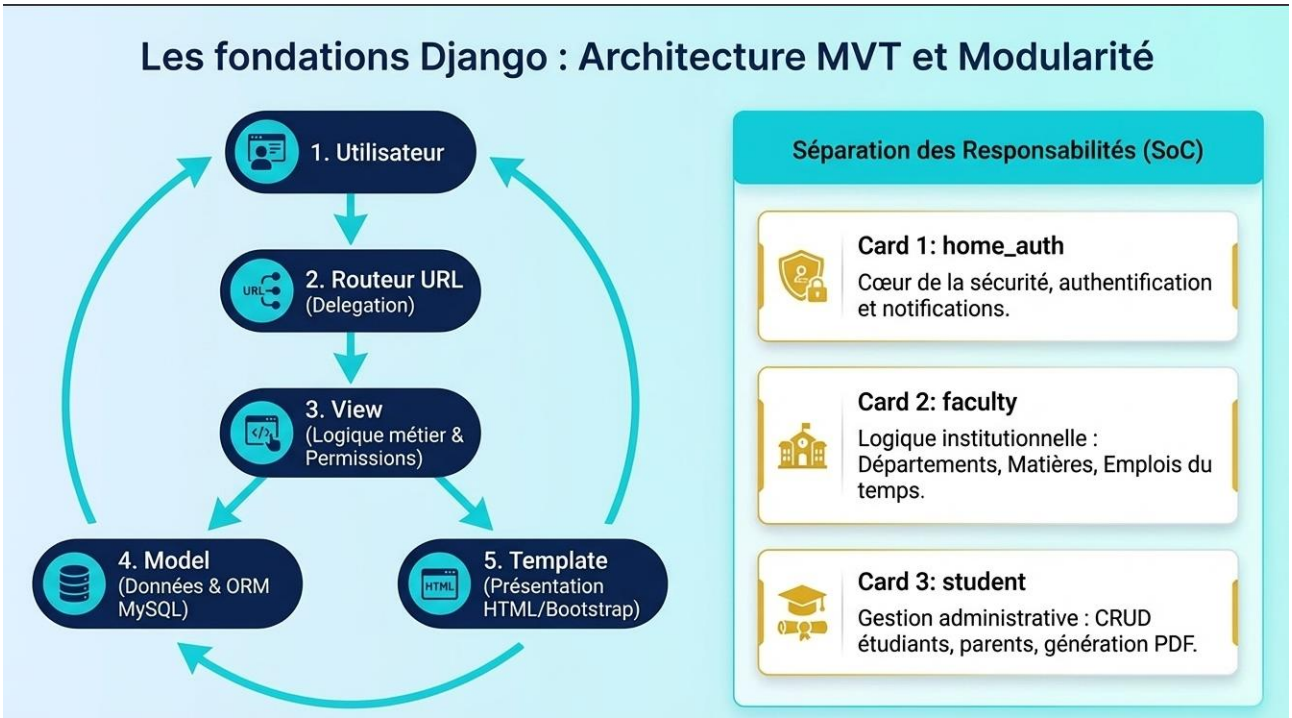
- Concevoir une architecture Django modulaire basee sur plusieurs applications independantes.
- Implementer un systeme d'authentification multi-role avec redirection conditionnelle.
- Integrer une base de donnees MySQL pour la persistance et l'integrite des donnees.
- Utiliser Bootstrap pour une interface responsive et moderne.

- Garantir la securite des acces via les decorateurs Django (login_required, permission_required).

3. Architecture Generale

3.1 Structure des Applications Django

Le projet Django PreSkool est organise selon le principe "une application = une responsabilite". Chaque module fonctionnel dispose de sa propre application Django, favorisant la modularite et la maintenabilite du code.

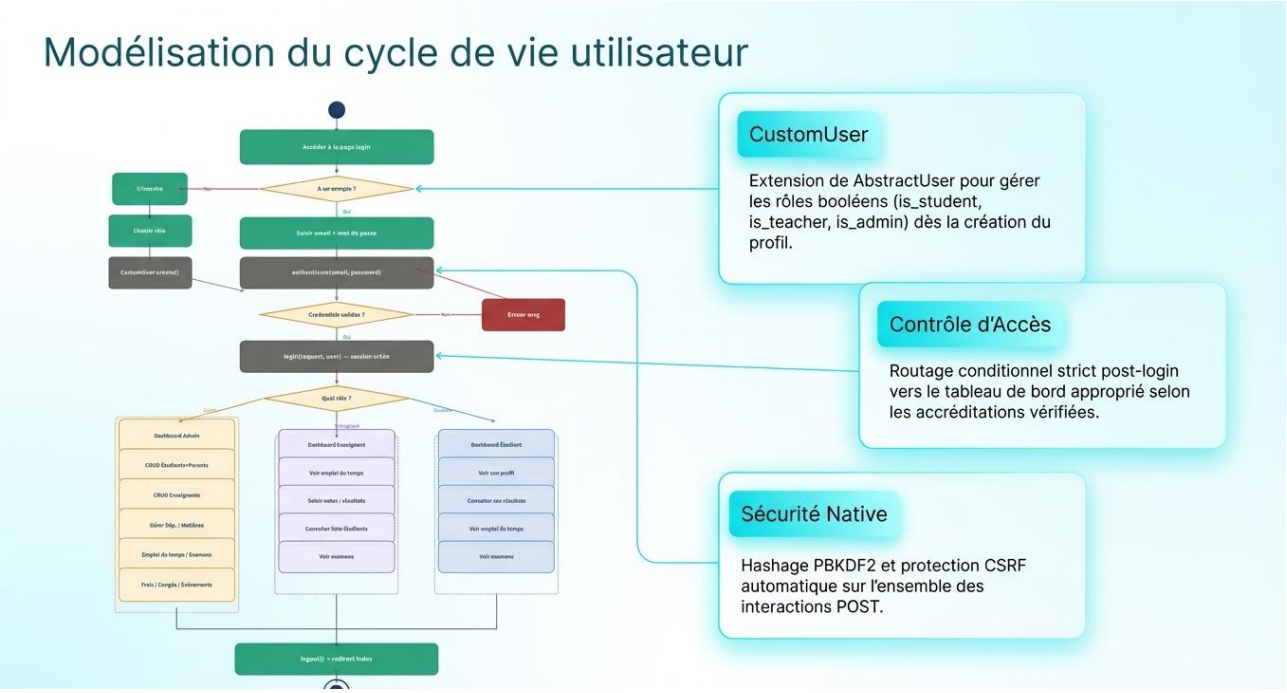


Le projet principal (school) orchestre les applications suivantes :

- home_auth : gestion de l'authentification, des roles et des sessions utilisateur.
- student : gestion des profils etudiants et des informations parentales.
- faculty : gestion des enseignants et de leurs affectations par departement.
- subject : gestion des matieres et des departements.
- exam : planification et gestion des examens, saisie des notes.
- timetable : generation et consultation des emplois du temps.
- fees : gestion des frais de scolarite et des factures.
- events : gestion des evenements et des jours feries.
- library : gestion des ressources documentaires.

3.2 Flux d'Authentification et de Navigation

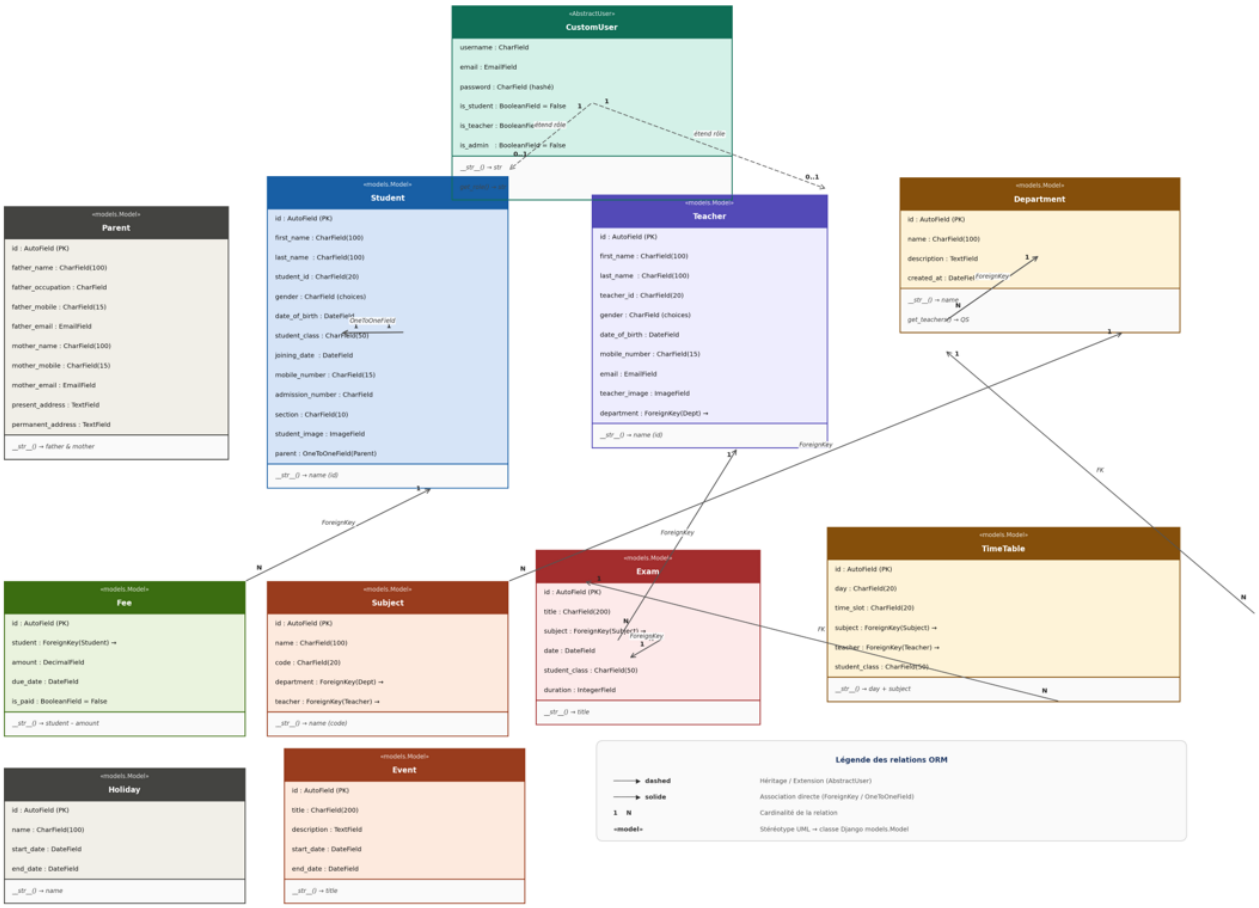
L'architecture suit le patron MVT (Model-View-Template) de Django. Chaque requete HTTP est routee via le fichier school/urls.py vers l'application concernee, puis traitee par la vue appropriee, qui interroge les modeles via l'ORM et renvoie le template HTML rendu.



4. Diagramme de Classes

4.1 Entites et Relations UML

Le modele de donnees de PreSkool Academy s'articule autour de dix entites principales, chacune materalisee par un modele Django heritant de models.Model. Les relations entre entites sont implementees via les champs relationnels de l'ORM Django (ForeignKey, OneToOneField, ManyToManyField).



4.2 Description des Entites Principales

CustomUser (AbstractUser) : entite centrale d'authentification. Elle etend le modele User natif de Django en ajoutant trois champs booleens (is_student, is_teacher, is_admin) qui determinent le role de chaque utilisateur dans le systeme.

Student : contient toutes les informations academiques et personnelles de l'etudiant (nom, prenom, date de naissance, filiere, numero de telephone, photo). Elle est liee a Parent par une relation OneToOneField et a CustomUser par une relation OneToOneField.

Teacher : profil de l'enseignant avec ses coordonnees, son identifiant et son departement de rattachement. Elle est egalement liee a CustomUser.

Department : represente un departement academique (Mathematiques, Informatique, Physique...) avec son nom et sa description. Les enseignants et les matieres lui sont rattaches via une ForeignKey.

Subject : matiere d'enseignement, reliee a un departement et potentiellement a un enseignant. Elle sert de point de jonction entre les examens et les emplois du temps.

Exam : entite de planification des epreuves. Chaque examen est associe a une matiere, une filiere, une date, un horaire et un statut (A venir, En cours, Termine).

TimeTable : entite representant un creneau horaire dans l'emploi du temps. Elle associe un jour, une plage horaire, une matiere et un enseignant.

Fee : gestion des frais de scolarite par etudiant, avec montant, echeance et statut de paiement (Paye / Impaye).

Holiday & Event : gestion du calendrier academique (jours feries officiels, evenements institutionnels).

5. Choix Techniques

5.1 Technologies et Justifications

Technologie	Justification
Django 4.x	Framework Python mature, batteries incluses : ORM, authentification, admin, systeme de templates. Ideal pour des applications CRUD complexes avec gestion des roles.
MySQL	SGBD relationnel robuste, adapte aux relations complexes entre entites. Support natif des transactions ACID, performances optimales pour les requetes multi-jointures.
Bootstrap 5	Framework CSS front-end permettant une interface responsive sans expertise CSS avancee. Grande bibliotheque de composants (tables, cards, navbar, graphiques).
Chart.js	Bibliotheque JavaScript legere pour la generation de graphiques interactifs (barres, camemberts, courbes) dans les tableaux de bord.
DataTables.js	Extension jQuery offrant des fonctionnalites avancees aux tableaux HTML : recherche, tri, pagination, export CSV/PDF/Excel.
WeasyPrint	Bibliotheque Python pour la generation de fichiers PDF a partir de templates HTML, utilisee pour les attestations et releves de notes.
Pillow	Bibliotheque Python de traitement d'images, utilisee pour la gestion des photos de profil des etudiants et enseignants.

python-decouple	Gestion propre des variables d'environnement (cle secrete, parametres de base de donnees) via un fichier .env pour la securite.

5.2 Justification du Choix de MySQL

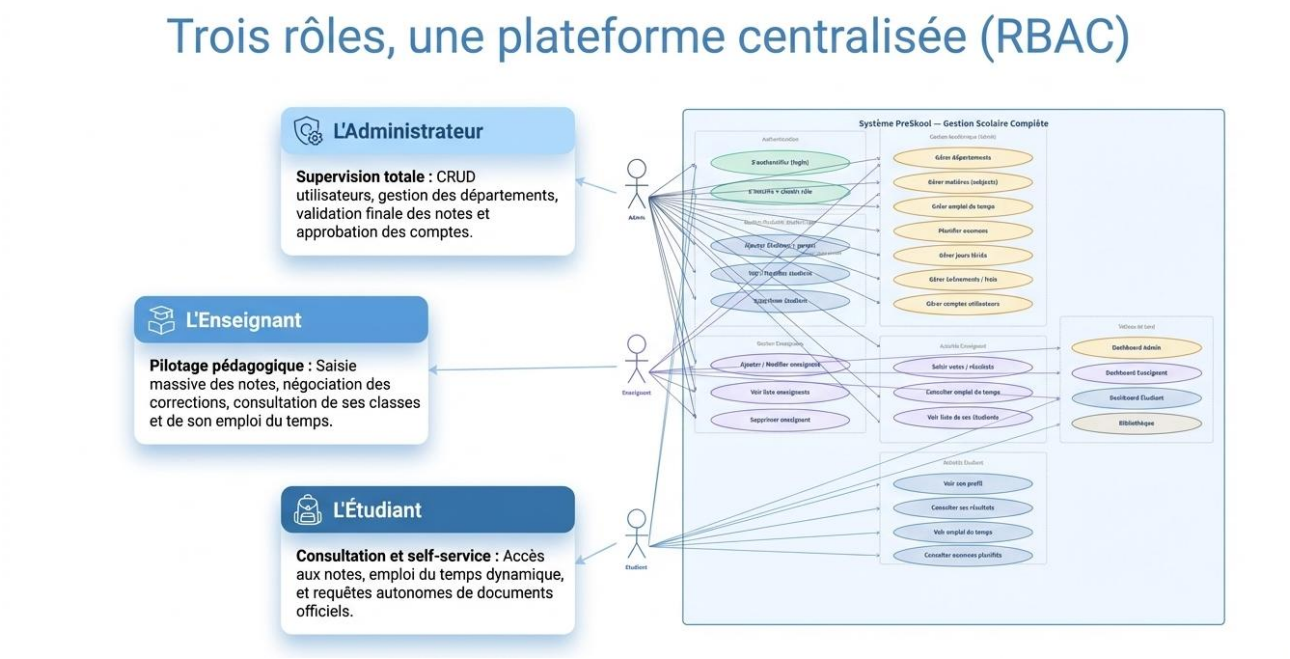
MySQL a ete prefere a SQLite (base par defaut de Django) pour plusieurs raisons :

- Support de la concurrence : MySQL gere efficacement les acces simultanes de plusieurs utilisateurs, indispensable dans un contexte scolaire.
- Integrite referentielle : les contraintes de cles etrangeres sont pleinement supportees, garantissant la coherence des donnees.
- Performances : les index et les optimisations de MySQL offrent de meilleures performances pour les requetes complexes impliquant plusieurs jointures.
- Readiness production : MySQL est un standard industriel, facilitant le deploiement sur des serveurs de production.

6. Description des Fonctionnalites Principales

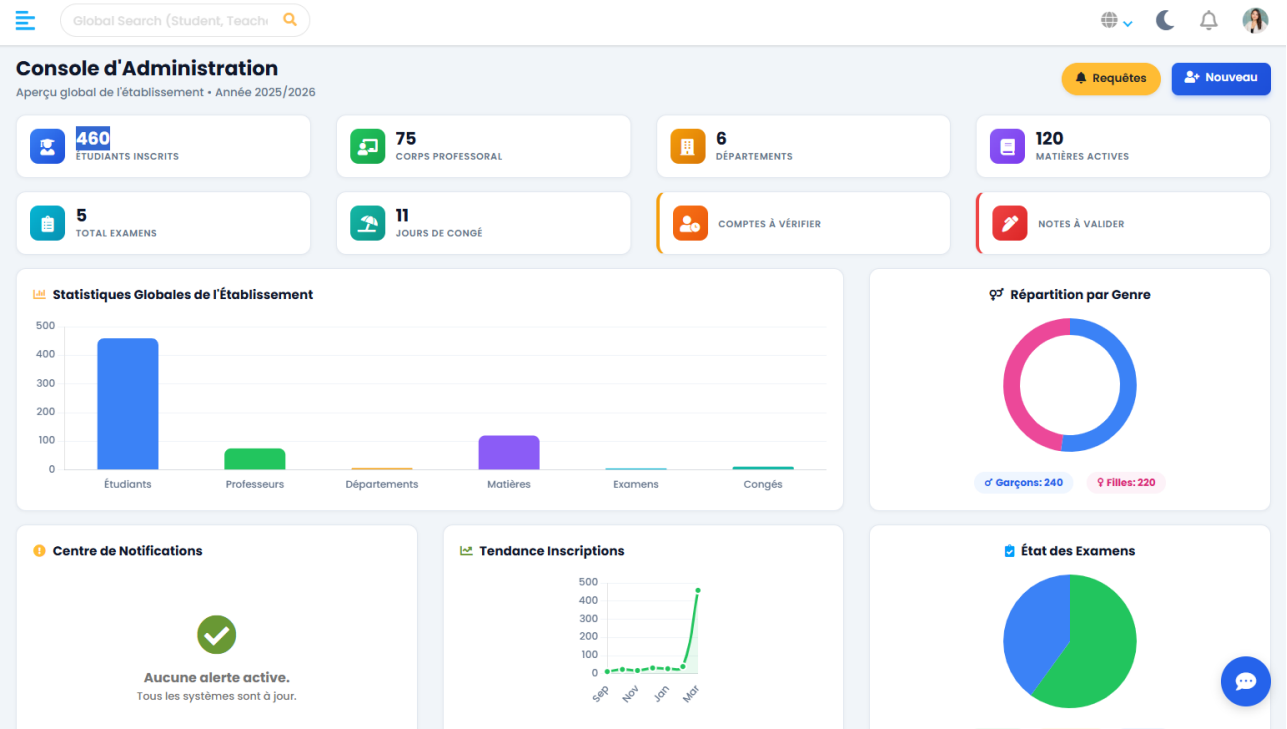
6.1 Authentification et Gestion des Roles

Le systeme d'authentification repose sur un modele CustomUser qui etend AbstractUser de Django. Lors de l'inscription, l'utilisateur choisit son role (Admin, Enseignant, Etudiant). Apres connexion, Django redirige automatiquement l'utilisateur vers le tableau de bord correspondant a son role.



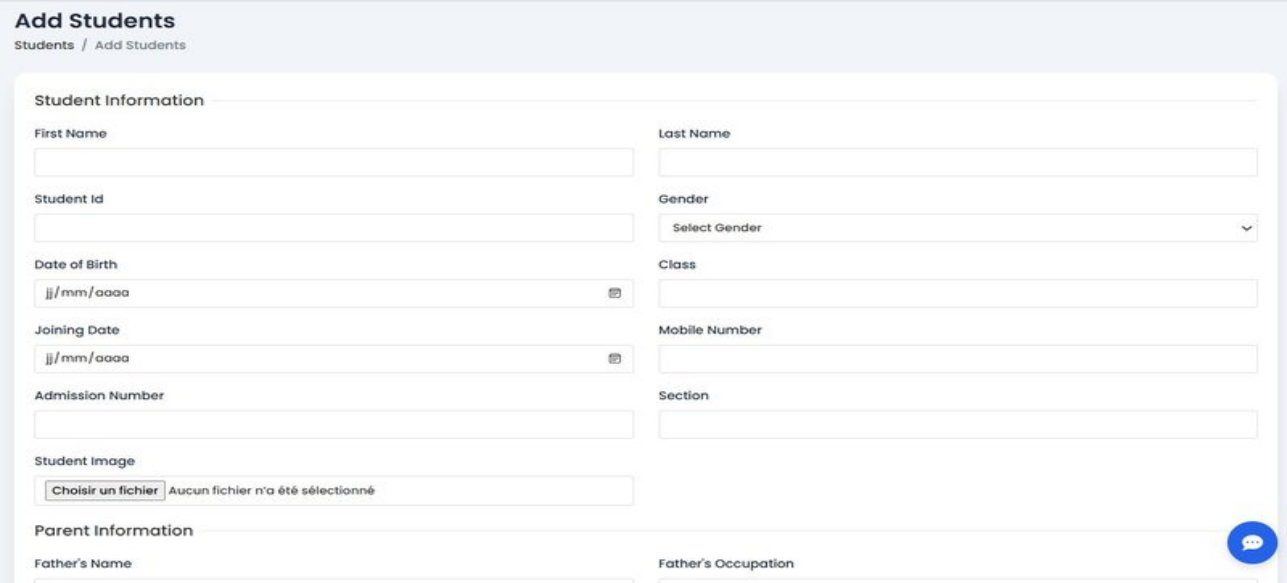
6.2 Console d'Administration

L'administrateur dispose d'une console de gestion complete offrant une vue globale de l'etablissement : nombre d'etudiants inscrits, corps professoral, departements, matieres actives, examens et jours de conge. Des graphiques statistiques (barres, camembert de repartition par genre) permettent une lecture rapide des indicateurs cles.



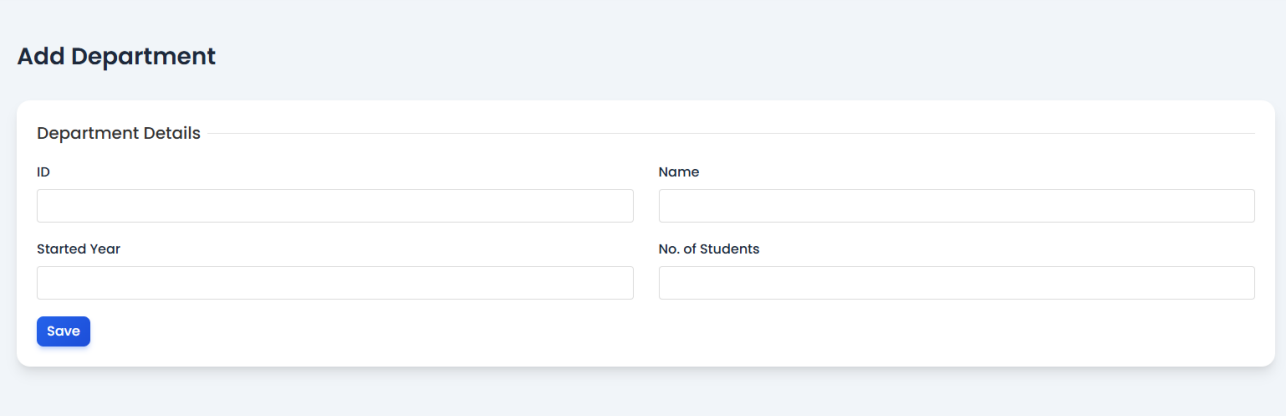
6.3 Gestion des Etudiants et Parents

L'administrateur peut ajouter, modifier ou supprimer des etudiants. Le formulaire d'ajout captuie les informations personnelles (nom, prenom, date de naissance, genre, filiere, photo) ainsi que les informations parentales (nom du pere, profession, coordonnees de la mere). Les deux entites Student et Parent sont creees atomiquement dans la base de donnees.



6.4 Gestion des Enseignants et Departements

L'application permet la creation de departements academiques et l'affectation d'enseignants a chacun. L'administrateur peut ajouter des matieres et les associer a un departement specifique. Chaque enseignant dispose d'un profil complet avec son identifiant, ses coordonnees et son rattachement departementiel.



Formulaire d'ajout d'une matiere avec selection du departement

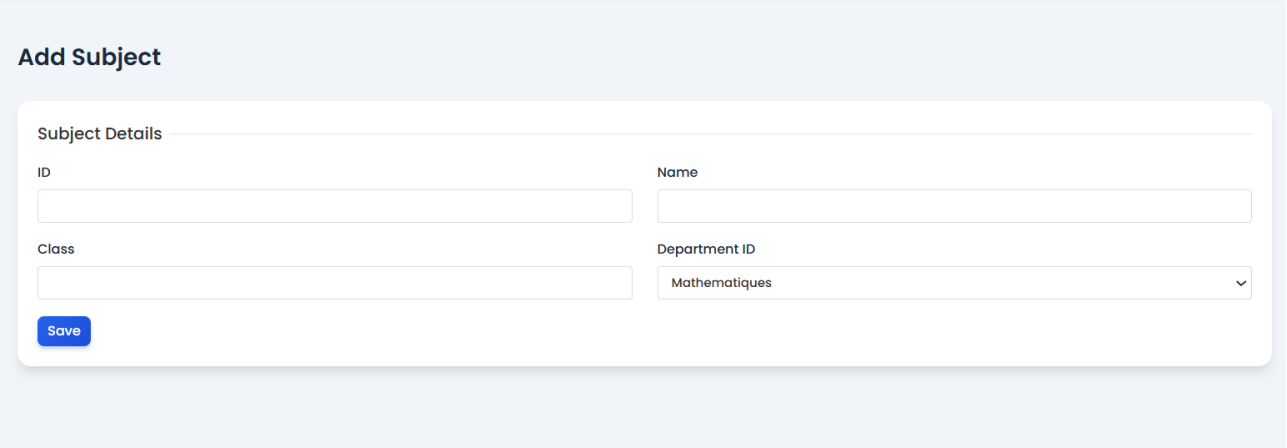
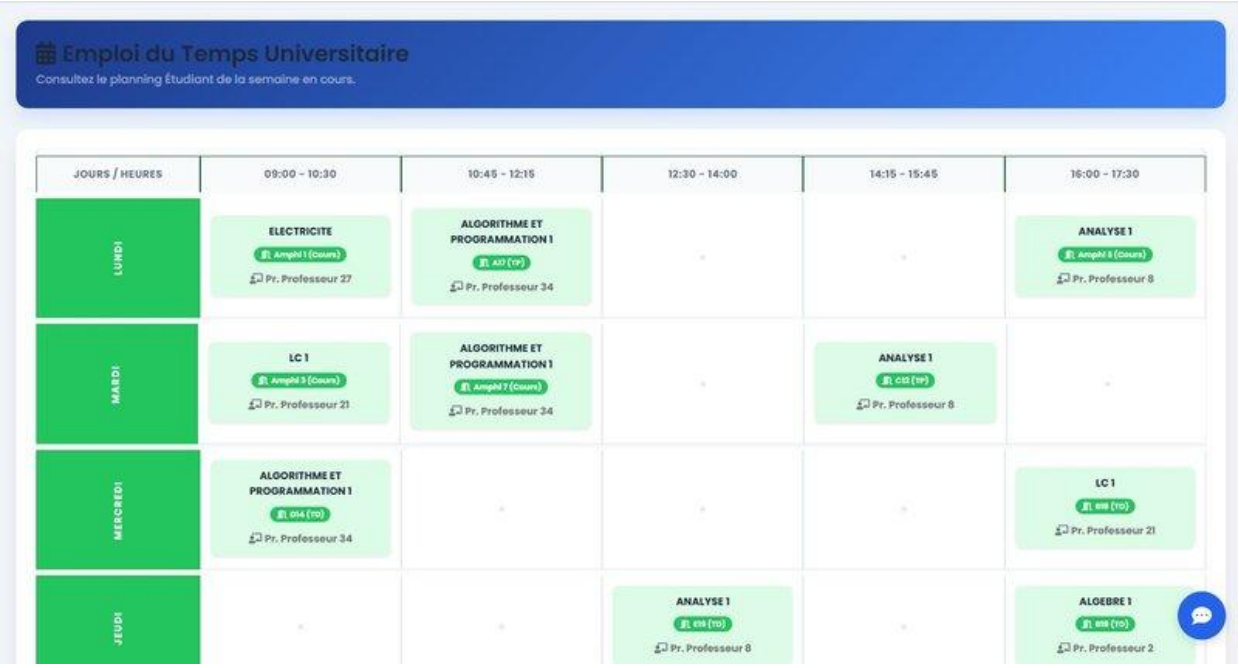


Figure 8 - Formulaire d'ajout d'un departement

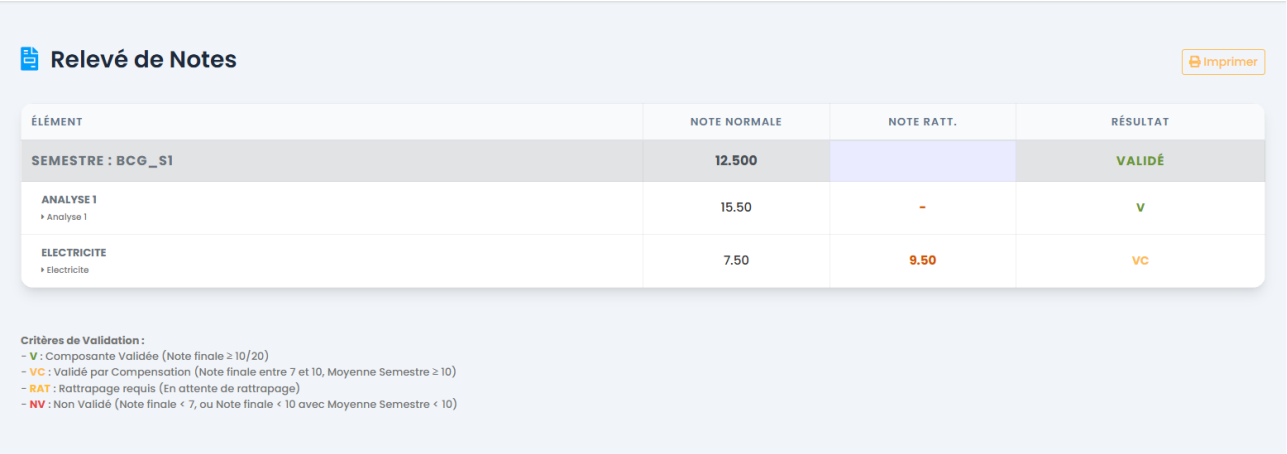
6.5 Emplois du Temps

Le module TimeTable permet la generation et la consultation des emplois du temps par filiere. Chaque creneau affiche la matiere, la salle, le type de seance (Cours, TD, TP) et l'enseignant responsable. Les etudiants et enseignants peuvent consulter leur planning hebdomadaire depuis leur espace personnel.



6.6 Gestion des Examens et des Notes

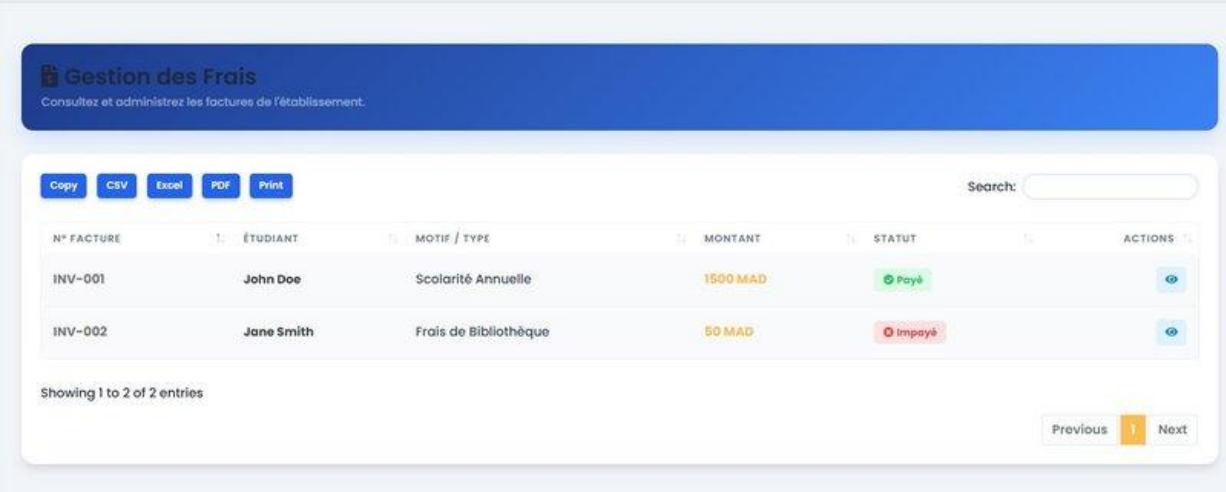
L'administrateur planifie les examens (matiere, filiere, date, horaire, points). Les enseignants saisissent les notes depuis leur espace. Un systeme de validation a deux niveaux (enseignant puis admin) garantit l'integrite des resultats. Le Bureau de Validation affiche l'historique de toutes les decisions d'approbation ou de refus de modification de notes.



Liste des notes avec statuts

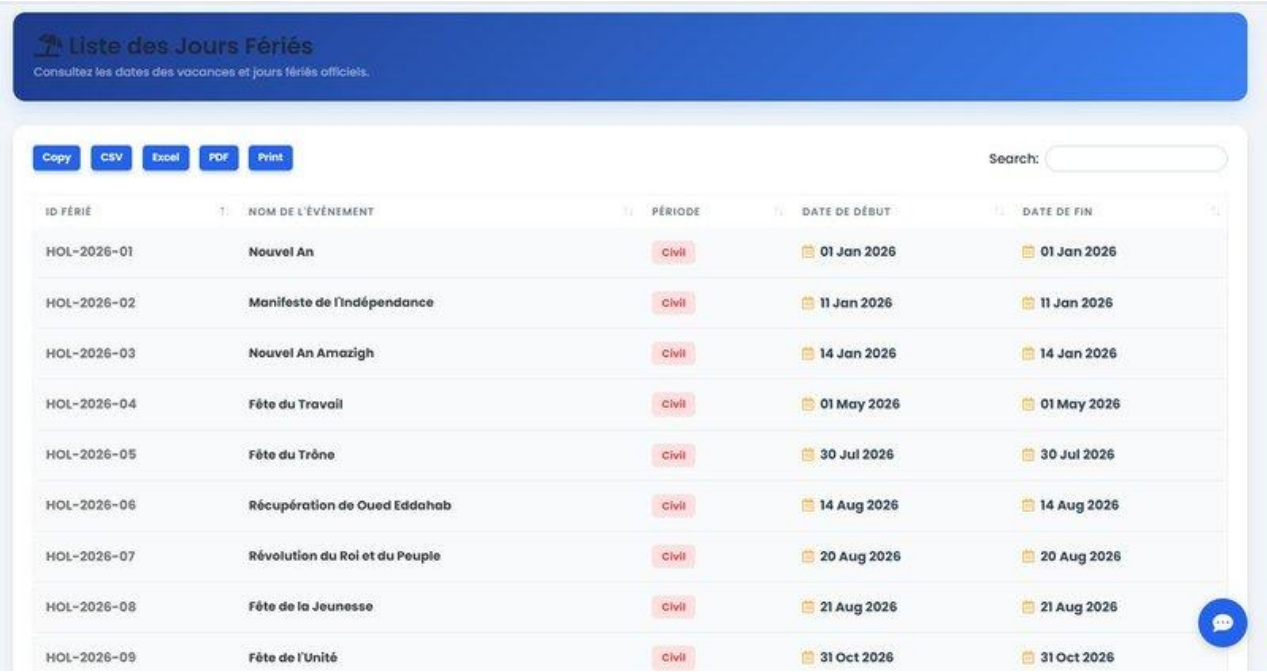
6.7 Gestion des Frais de Scolarite

Le module Fees permet de suivre les frais de scolarite de chaque etudiant. Chaque facture est associee a un etudiant, un motif (Scolarite annuelle, Frais de bibliotheque...), un montant en MAD et un statut (Paye / Impaye). L'interface dispose de fonctionnalites d'export en CSV, Excel et PDF.



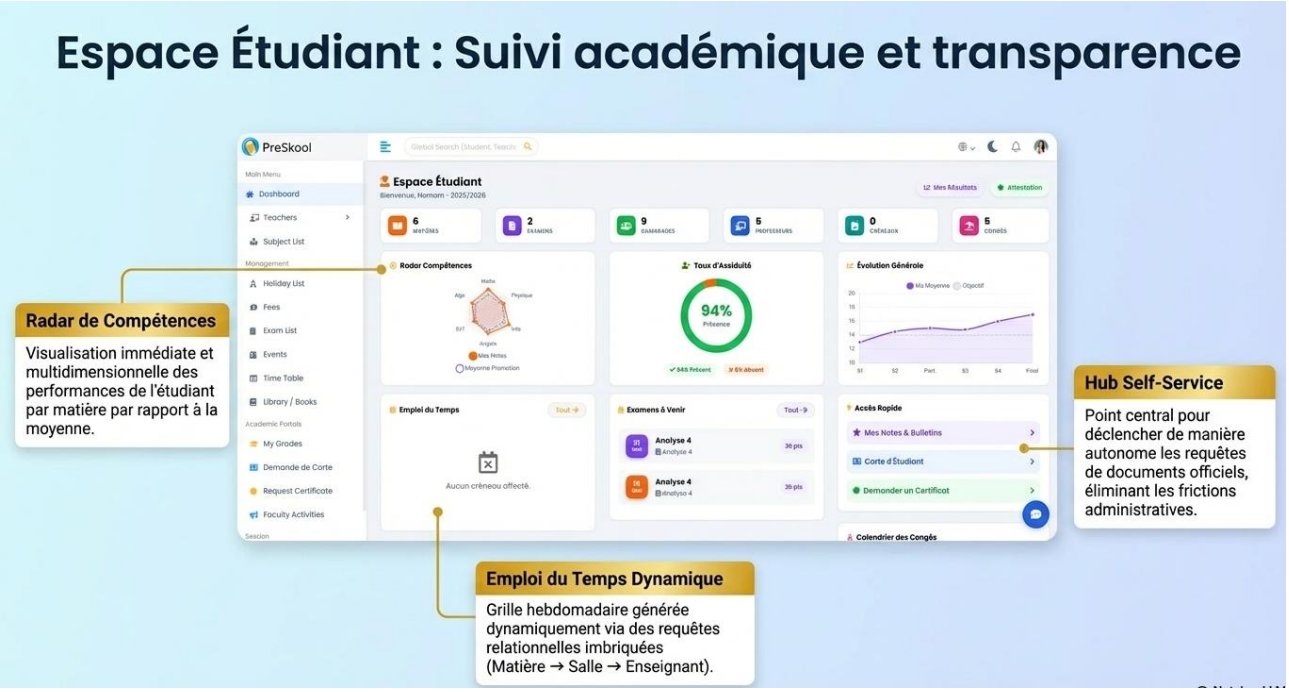
6.8 Gestion des Jours Feries et Conges

L'application integre une liste des jours feries officiels du Maroc pour l'annee academique. L'administrateur peut ajouter, modifier ou supprimer des entrees. Les etudiants et enseignants peuvent consulter ce calendrier depuis leur espace personnel.



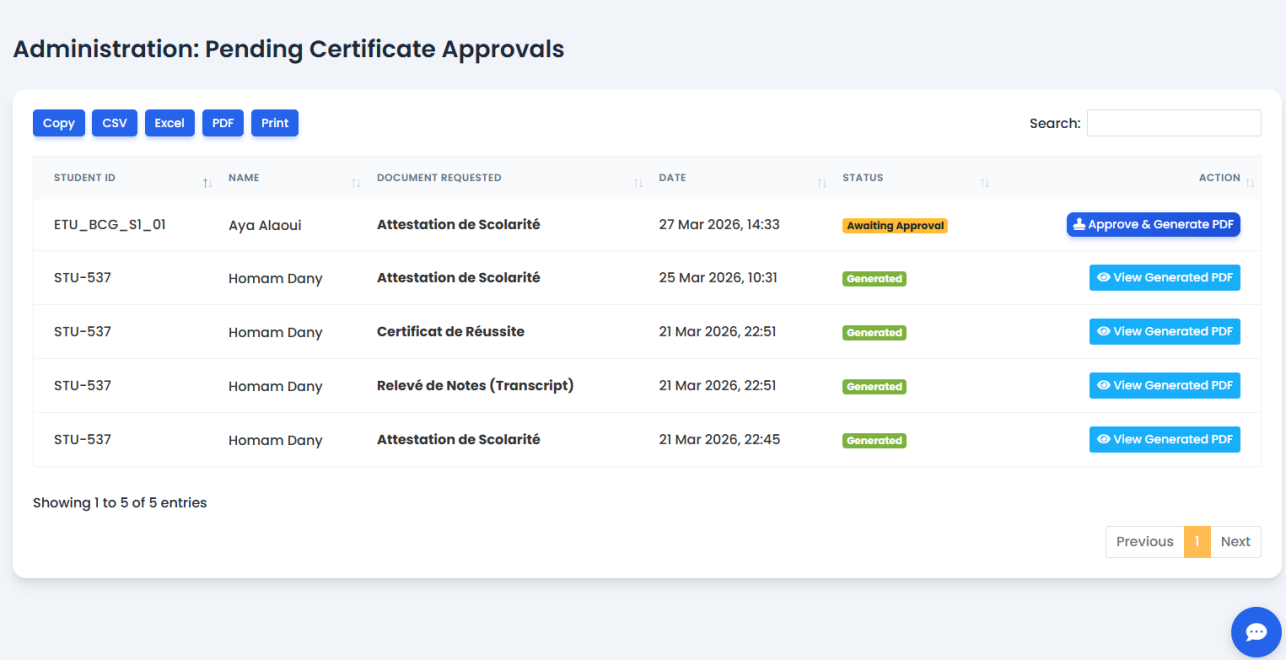
6.9 Espace Etudiant et Tableau de Bord

Chaque etudiant dispose d'un espace personnel avec un tableau de bord complet : radar de competences par matiere, taux d'assiduite, evolution de la moyenne semestre par semestre, examens a venir et acces rapide aux services (notes, carte etudiant, certificats).



6.10 Demande et Validation de Certificats

Les etudiants peuvent demander en ligne des documents officiels : Attestation de Scolarite, Releve de Notes (Transcript) ou Certificat de Reussite. Apres validation par l'administration, le document est genere en PDF et mis a disposition pour telechargement direct depuis l'espace etudiant.



Services Avancés 1/2 : Automatisation documentaire instantanée

xhtml2pdf & Django Templates

Conversion directe des vues HTML/CSS en flux binaire PDF, conservant le design institutionnel.

Le Défi des Chemins (link_callback)

Implémentation d'une fonction Python sur mesure pour résoudre les URL relatives de Django en chemins absolus locaux (nécessaire pour l'inclusion sécurisée des logos et signatures).

Composition avec Pillow

Traitement d'image dynamique utilisé en parallèle pour la génération des cartes d'étudiant avec intégration de la photo d'identité.

PRESKOOL ACADEMY

DÉPARTEMENT ADMINISTRATIF & SCOLARITÉ UNIVERSITAIRE

ATTESTATION DE SCOLARITÉ

Le directeur de l'établissement d'enseignement supérieur certifie par la présente que :

L'étudiant(e) **HOMAM DANY**,
Né(e) le **01 Jan 2000**,
Inscrit(e) sous le numéro d'immatriculation officiel **STU-537**.

Est régulièrement inscrit(e) et poursuit assidûment ses études au sein de notre établissement pour l'année universitaire de la promotion en cours, au sein de la classe **MIP_S4**.

Ce document est délivré à l'intéressé(e) pour servir et valoir ce que de droit, et atteste de son inscription formelle dans nos registres académiques.

OFFICIEL

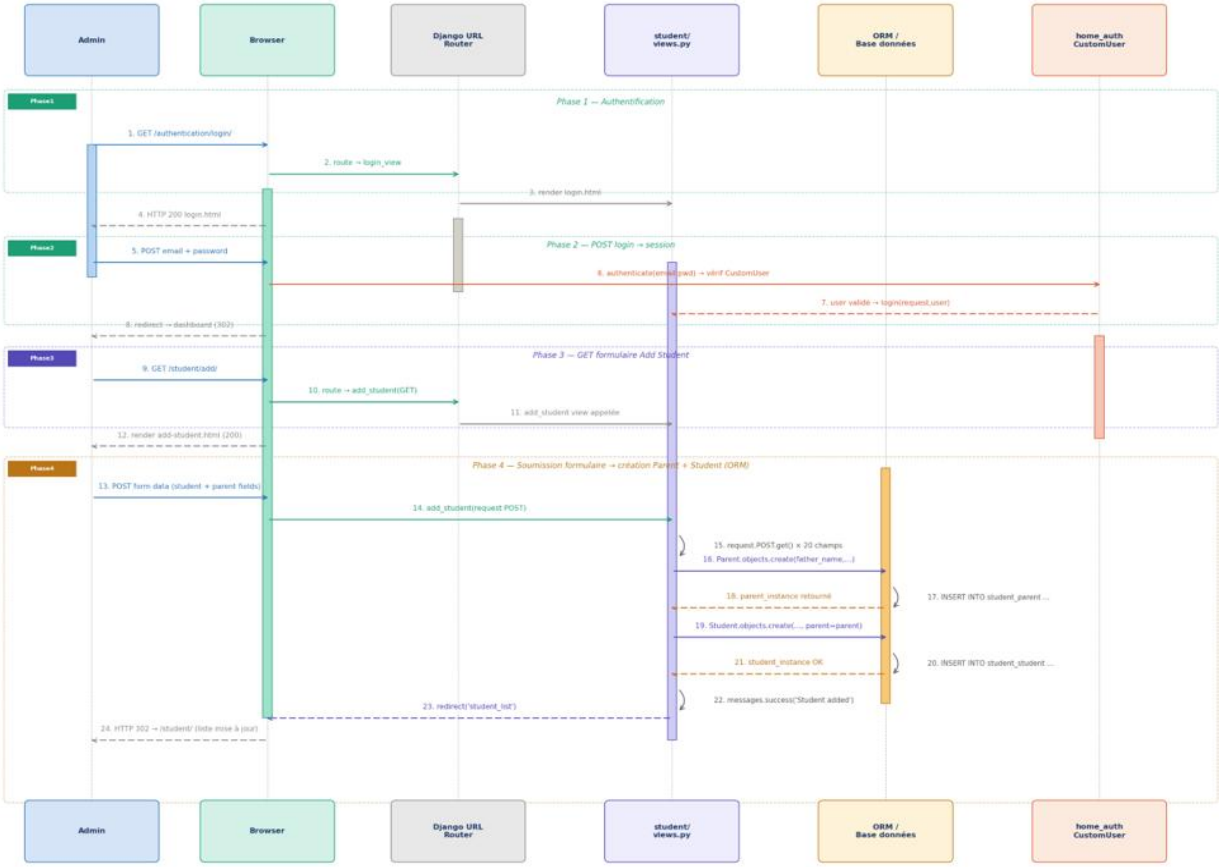
Fait à Paris, le 28 Mar 2026

La Direction Générale

7. Diagrammes UML Complementaires

7.1 Diagramme de Sequence

Le diagramme de sequence ci-dessous illustre le flux complet d'une session administrateur : depuis l'authentification jusqu'a la creation d'un etudiant, en passant par toutes les interactions entre les composants Django (URL Router, Views, ORM, Base de donnees).



8. Difficultes Rencontrees

8.1 Configuration de l'Environnement de Developpement

La mise en place de l'environnement virtuel Python et l'installation des dependances a constitue un premier defi. Les conflits de versions entre Django, mysqlclient et les bibliotheques tierces ont necessite une gestion rigoureuse du fichier requirements.txt et l'utilisation de pip freeze pour verrouiller les versions.

- Erreur : "django.db.utils.OperationalError: no such table" apres des migrations non appliquees. Solution : execution systematique de python manage.py makemigrations puis migrate.
- Probleme de connexion MySQL : le driver mysqlclient necessitait l'installation prealable des headers C de la bibliotheque libmysqlclient-dev sur Linux.

8.2 Gestion des Relations ORM Complexes

La relation OneToOne entre CustomUser, Student et Teacher a presente des difficultes lors de la creation atomique des deux entites. Une exception ObjectDoesNotExist etait levee lorsque l'acces au profil etudiant precedait la creation du compte utilisateur.

- Solution : utilisation de get_or_create() et de blocs try/except pour les relations conditionnelles.
- Gestion du signal post_save pour la creation automatique du profil lors de l'enregistrement d'un CustomUser.

8.3 Generation de PDF

L'integration de WeasyPrint pour la generation de certificats PDF a souleve des problemes de compatibilite avec les polices systeme et les images embarquees. Les chemins absolus vers les fichiers statiques devaient etre references correctement dans les templates HTML utilises comme source.

- Solution : utilisation de request.build_absolute_uri() pour resoudre les URLs des images dans les templates PDF.
- Configuration de STATICFILES_DIRS et de MEDIA_ROOT pour garantir l'accessibilite des ressources statiques depuis WeasyPrint.

8.4 Securite et Controle d'Acces

La protection des vues selon les roles utilisateur a necessite une reflexion approfondie sur l'utilisation des decorateurs Django. Certaines vues accessibles aux enseignants ne devaient pas l'etre aux etudiants, meme si ces derniers etaient authentifies.

- Solution : creation de decorateurs personnalises (@admin_required, @teacher_required, @student_required) encapsulant la logique de verification du role.
- Tests systematiques des URLs avec differents comptes pour valider le cloisonnement des acces.

9. Ameliorations Possibles

9.1 Fonctionnalites Non Implementees

Plusieurs fonctionnalites identifiees lors de la conception n'ont pas pu etre implementees dans le delai imparti :

- Messagerie interne : un systeme de communication direct entre etudiants, enseignants et administration, avec notifications en temps reel via WebSockets (Django Channels).
- Application mobile : une version mobile native (React Native ou Flutter) consommant une API REST Django (Django REST Framework) pour offrir une experience optimisee sur smartphone.
- Systeme de paiement en ligne : integration d'une passerelle de paiement (CMI, PayPal) pour le reglement en ligne des frais de scolarite.
- Gestion des absences : un module de suivi des presences par seance, avec alertes automatiques en cas de depassement du seuil d'absences.
- Module de reclamations : un espace structure permettant aux etudiants de soumettre des reclamations concernant leurs notes.

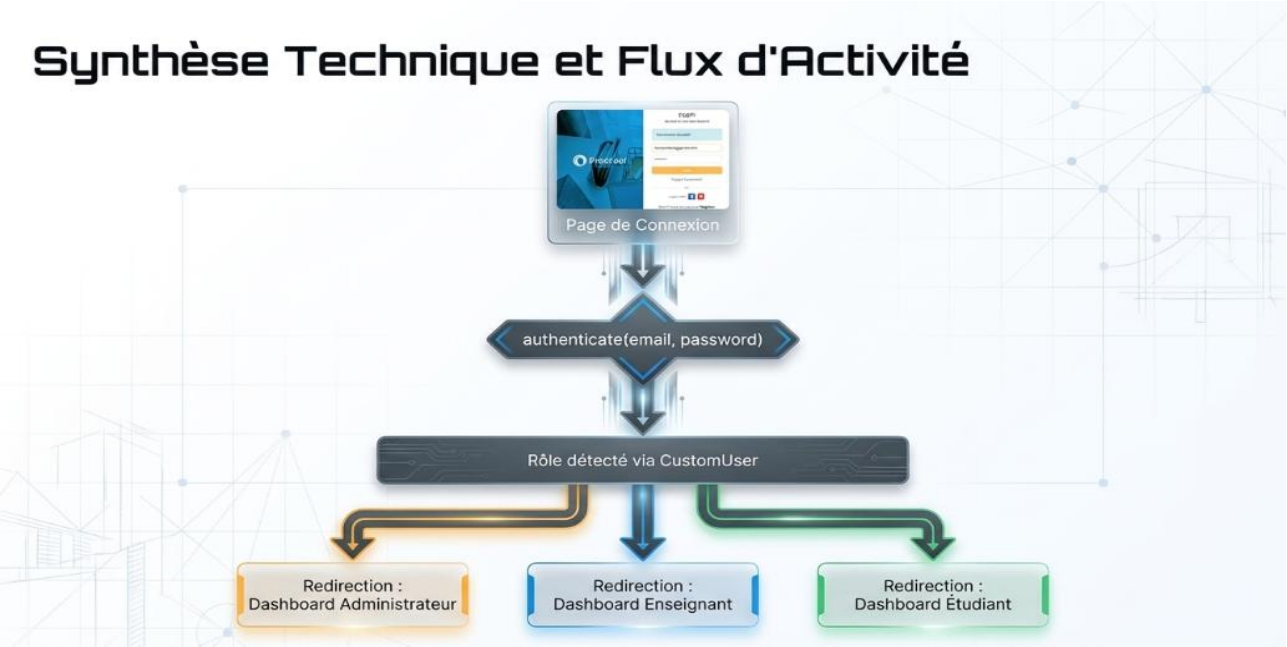
9.2 Optimisations Techniques

- Mise en cache : implementation de Django Cache Framework (Redis) pour reduire les requetes redondantes a la base de donnees sur les tableaux de bord.
- Tests automatises : ecriture d'une suite de tests unitaires et d'integration (pytest-django) pour garantir la stabilite du code lors des evolutions futures.
- API REST : exposition d'une API RESTful via Django REST Framework pour permettre l'integration avec des systemes tiers (ENT, LDAP universitaire).
- Internationalisation (i18n) : extension du support multilingue (arabe, anglais) en utilisant les outils de traduction integres de Django.
- Deploiement : containerisation de l'application avec Docker et Nginx/Gunicorn pour faciliter le deploiement en production et la scalabilite horizontale.
- Optimisation des requetes : utilisation systematique de select_related() et prefetch_related() pour reduire le nombre de requetes SQL generees par l'ORM.

9.3 Conclusion

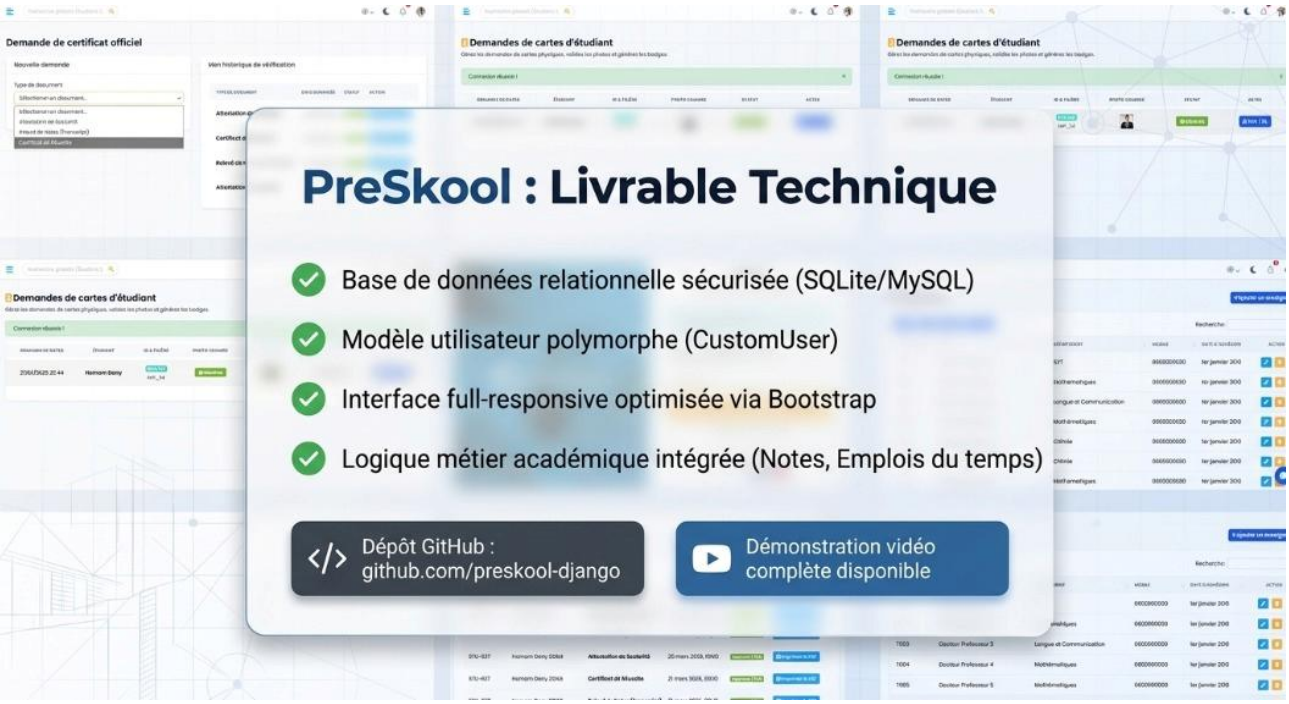
Le projet PreSkool Academy nous a permis d'acquerir une maitrise approfondie du framework Django et de ses ecosystemes (ORM, templates, authentification, middleware). Au-dela des aspects purement techniques, ce projet nous a confrontes a des problematiques reelles de gestion d'etablissement scolaire, renforçant notre capacite a analyser un besoin metier et a le traduire en solution informatique structuree.

L'application realisee constitue une base solide et extensible sur laquelle des fonctionnalites supplementaires pourront etre greffees. Elle demontre la pertinence de Django comme framework de choix pour le developpement d'applications web institutionnelles exigeant securite, modularite et rapidite de developpement.



10. Lien de GitHub

Je me permets de vous partager le lien de notre dépôt GitHub contenant l'ensemble du travail réalisé pour le projet.
Dans ce dépôt, vous trouverez :

- Une **documentation README** détaillée et rédigée de manière professionnelle, expliquant le projet et son fonctionnement.
- Le **code source complet de l'application**.
- Une **vidéo démonstrative** présentant le fonctionnement de l'application et ses principales fonctionnalités.

Vous pouvez consulter le projet via le lien suivant :
[https://github.com/Homam-Dany/Application_Web_Systeme_De_Gestion_Scolaire]